Clase 28 — VLSM y planes de numeración

Bloque: D (Capa de Red) · **Semana:** 10 · **Duración estimada:** 100 min **Correlativa:** Clase 27 (subnetting paso a paso) · **Sigue con:** Clase 29 (taller de subnetting y planes de numeración) **Material de referencia:** Tanenbaum cap. 5 · RFC 1918 (privadas) 🔍 · RFC 4632 (CIDR) 🔍 · RFC 3021 (/31 P2P) 🔍 · RFC 6177 · RFC 8200

> 🔍 = RFC **no verificado** en `ACTUALIZACIONES.md` : no lo cites sin confirmarlo.

🎯 Al terminar esta clase vas a poder

1. Explicar por qué el subnetting clásico (FLSM) desperdicia direcciones.
2. Calcular el prefijo de una subred a partir de una cantidad de hosts: `2^h − 2 ≥ H`.
3. Aplicar el procedimiento de VLSM: ordenar de mayor a menor, asignar **contiguo y alineado**, llevar el puntero.
4. Demostrar **por qué** hay que alinear, con la cuenta en AND de la máscara.
5. Distinguir "maximizar el uso **hoy**" de "maximizar lo que queda **para el futuro**", y **verificar** una solución oficial en vez de copiarla.
6. Diseñar un plan de numeración completo (casa central + sucursales) con su justificación.

Como se aplica una mascara de subred, bit a bit

IP	Mascara (/26)	AND = resultado
11000000	11111111	**11000000**
10101000	11111111	**10101000**
00001010	11111111	**00001010**
00110111	11000000	**00000000**

192.168.10.55 AND 255.255.255.192 = 192.168.10.64

Lo que resulta de ese AND:

direccion de red	192.168.10.64
primer host	192.168.10.65
ultimo host	192.168.10.126
broadcast	192.168.10.127
hostables	62

El prefijo /26 significa:

```
26 bits de red + 6 bits de host
2^6 = 64 direcciones en la subred
64 - 2 = 62 hostables

El AND pone en 0 todos los bits
de host: por eso el resultado es
la direccion de RED, no una de host.
```

> **Prerequisito del examen:** `2011-03 P2d` · `2010-12 P2a-i` · `2010-07 P2a-i` · `2006-08 P4` · `2008-12 P2a-i`.

PARTE 1 — TEORÍA

1.1 El problema que resuelve VLSM

La clase 27 te dio el método: agarrás un bloque, decidís cuántas subredes querés, robás bits de host y quedás con **todas del mismo tamaño**. Eso es **FLSM** (*Fixed Length Subnet Mask*), que sirve para una sola cosa: **repartir un bloque en partes iguales**. En cuanto las partes **no** son iguales, obliga a que todas sean tan grandes como la mayor, y ahí empieza el desperdicio.

La aritmética del desastre

La empresa recibe `172.16.0.0/20` (4096 direcciones) y necesita dos LAN: **LAN-A** con 1000 puestos + 1 router = **1001** direcciones, y **LAN-B** con 100 puestos + 1 router = **101**.

Con FLSM todas las subredes deben poder alojar la más exigente, así que todas son `/22` (1024 direcciones). Y la LAN chiquita, que necesitaba 101, recibe 1024:

```
2^h − 2 ≥ 1001  →  2^10 − 2 = 1022 ≥ 1001  →  h = 10  →  prefijo = 32 − 10 = /22

LAN-A  172.16.0.0/22   1024 direcciones   (pedía 1001)   desperdicia  23
LAN-B  172.16.4.0/22   1024 direcciones   (pedía  101)   desperdicia 923
                        ────────                                  ───────
Direccionadas:          2048                          desperdicio:  946  ≈ 1000
Libre del /20:          2048
```

Ahí están **las ~1000 direcciones desperdiciadas** del curso: el 89 % de la LAN-B está sin usar.

Con VLSM cada subred toma **justo** lo que necesita:

```
LAN-A 1001 → 2^10 − 2 = 1022 ≥ 1001  →  /22   1024 dir
LAN-B  101 → 2^7  − 2 =  126 ≥  101  →  /25    128 dir
                        ────────              ──────
Direccionadas:          1152              desperdicio: 23 + 27 = 50
Libre del /20:          2944

Ahorro:  2048 − 1152 = 896 direcciones (44 % menos)
         946 − 50 = 896 menos desperdicio (95 % menos)
```

Y lo que **no** cambió: la LAN-B necesita 101 y 50 de sobra es el **piso teórico** — siempre quedan la dirección de red, la de broadcast y la diferencia hasta la potencia de 2 más cercana. VLSM te acerca a ese piso.

> **VLSM** (*Variable Length Subnet Mask*, máscara de longitud variable): método de subnetting en el que **cada subred usa un prefijo distinto**, elegido según los hosts que realmente necesita. Es **el método correcto** para maximizar el uso de direcciones.

1.2 El procedimiento de VLSM, paso a paso

```
PASO 1 — LISTAR las subredes de MAYOR a MENOR cantidad de hosts.
          Es OBLIGATORIO: asignar primero los bloques grandes es lo que
          garantiza que todos entren sin huecos.
PASO 2 — Por cada una:  2^h − 2 ≥ hosts   →   h = ceil(log2(H + 2))
          → prefijo = 32 − h.  Anotá prefijo y tamaño (2^(32 − prefijo)).
PASO 3 — ASIGNAR los bloques de mayor tamaño primero, de forma CONTIGUA y
          ALINEADA. Alineada = la dirección de red elegida es múltiplo del
          tamaño del bloque.
PASO 4 — Llevar la cuenta del PUNTERO: arranca en la dirección de red del
          bloque original y se le suma el tamaño de cada bloque.
```

El paso 1 en un ejemplo. `192.168.1.0/24` para LAN de 100 hosts, de 30, de 10, y dos enlaces punto a punto. El orden correcto es `/25 · /26 · /27 · /30 · /30`. Si el enunciado viniera en el orden `LAN 10 / enlace / LAN 30 / LAN 100` y asignás el `/28` primero, el puntero avanza 16 y el `/25` tiene que arrancar en el siguiente múltiplo de 128: como 16 no es múltiplo de 128, el `/25` se corre a `.128` y queda un hueco de **112 direcciones**.

1.3 Por qué hay que alinear (la cuenta, a la vista)

Ya asignaste `192.168.1.128/27` (32 direcciones: `.128`–`.159`) y el puntero quedó en `.160`. Necesitás un `/26`. El error fácil es arrancar en `.160`:

```
¡ERROR!  192.168.1.160/26  NO es una dirección de red válida.

/26 = 255.255.255.192 = 11111111.11111111.11111111.11000000
Los últimos 6 bits deben ser CERO en una dirección de red.

   160 = 10100000
  .192 = 11000000      ← los 6 bits bajos en cero
   AND
   160 AND 192 = 10100000 AND 11000000 = 10000000 = 128

Si numeraras una red en .160/26, en realidad estarías numerando
192.168.1.128/26, que SE SUPERPONE con el /27 que ya te comiste.
```

> **La regla:** la dirección de red de un bloque de tamaño `S` tiene que ser **múltiplo de** `S`: `dirección_de_red mod S == 0`.

La pregunta correcta es: ¿cuál es el **primer múltiplo de 64 mayor o igual que 160**? `160 / 64 = 2,5` → redondear hacia arriba → `3` → `3 × 64 = 192`.

```
192.168.1.128/27  →  .128 – .159   (32 dir)
──────────── hueco descartado: .160 – .191  (32 dir) ────────────
192.168.1.192/26  →  .192 – .255   (64 dir)
```

El puntero, con la cuenta completa

`192.168.1.0/24`, bloques de mayor a menor: `/25 (128) · /26 (64) · /27 (32) · /30 (4) · /30 (4)`

```
puntero = 192.168.1.0
/25  tamaño 128   inicio = menor múltiplo de 128 ≥ 0    = 0     →  .0/25   (  .0 – .127)
      puntero = 0 + 128 = 128
/26  tamaño  64   inicio = menor múltiplo de  64 ≥ 128  = 128   → .128/26  (.128 – .191)
      puntero = 128 + 64 = 192
/27  tamaño  32   inicio = menor múltiplo de  32 ≥ 192  = 192   → .192/27  (.192 – .223)
      puntero = 192 + 32 = 224
/30  tamaño   4   inicio = menor múltiplo de   4 ≥ 224  = 224   → .224/30  (.224 – .227)
      puntero = 224 + 4 = 228
/30  tamaño   4   inicio = menor múltiplo de   4 ≥ 228  = 228   → .228/30  (.228 – .231)
      puntero = 228 + 4 = 232

192.168.1.0/24
├─ 0     /25   LAN de 100 hosts      ├─ 224   /30   enlace P2P
├─ 128   /26   LAN de 30 hosts       ├─ 228   /30   enlace P2P
├─ 192   /27   LAN de 10 hosts       └─ 232 ──────── LIBRE (24) = 232/29 + 240/28

Verificación:  128 + 64 + 32 + 4 + 4 = 232        256 – 232 = 24  ✓
```

> **Atajo para el examen:** si los tamaños van **siempre de mayor a menor**, cada tamaño divide al anterior (256 | 128 | 64 | 32 | … | 4), así que el puntero **siempre** queda alineado y **nunca** hay huecos. Los huecos aparecen solo cuando asignás un bloque grande **después** de uno chico, o cuando los tamaños no forman una cadena de divisores. Por eso el paso 1 es obligatorio.

1.4 El objetivo del examen: "maximizar para el futuro"

El enunciado más repetido de la cátedra, textual del `2011-03 P2d`:

> "…que permitan **dejar la mayor cantidad posible de direcciones IP para uso futuro**."

Tiene dos readings, y el professor puede estar preguntando cualquiera:

```
LECTURA A – "gastá lo mínimo hoy"
Minimizar las direcciones consumidas.  ⇔ Elegir el prefijo MÁS CORTO que
todavía alcance para los hosts.

LECTURA B – "dejá el bloque libre más limpio"
Maximizar el mayor prefijo contiguo libre.  ⇔ Asignar contiguo y alineado,
para que el remanente no quede partido en pedacitos inservibles.
```

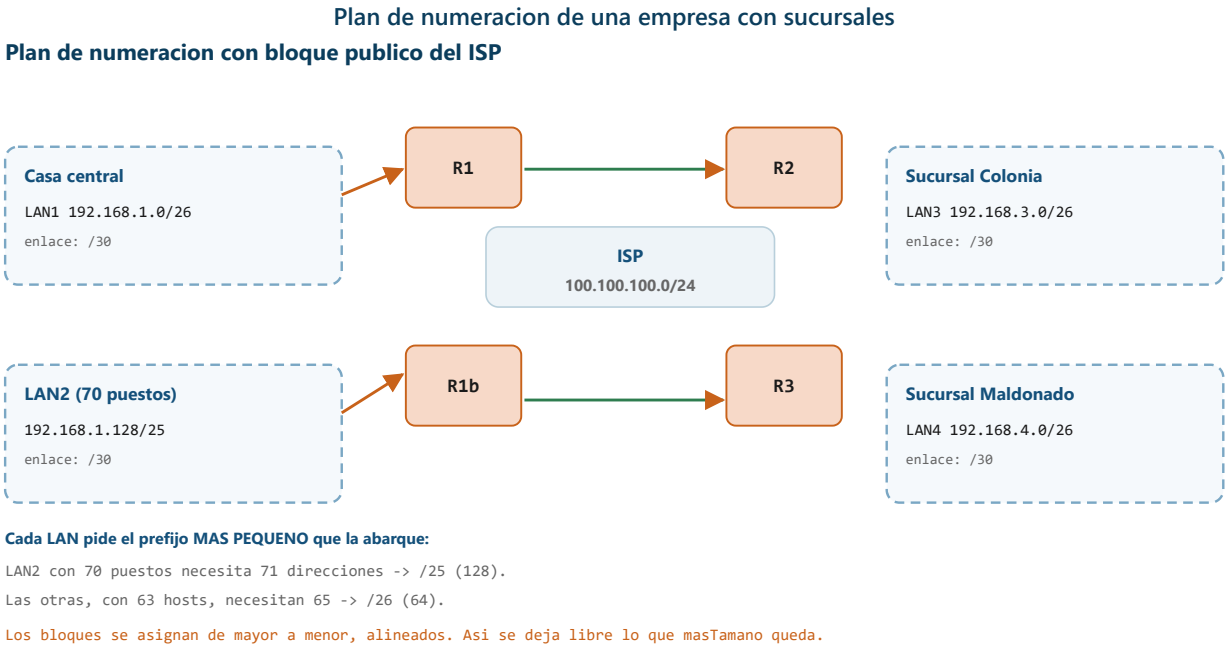
Las dos se responden juntas, y por eso el procedimiento es el mismo. Lo que cambia es el **orden de prioridad**: entre un plan que gasta menos hoy pero parte el remanente y otro que gasta un poco más pero deja un bloque limpio, la lectura B prefiere el segundo. En los ejercicios de la cátedra casi siempre coinciden: **decí las dos cosas**.

1.5 ⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - `2^h − 2 ≥ H` y la mecánica de robar bits de host. - Los tres rangos privados: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. **Cuidado:** `/12` no es `/16` — de `172.16.0.0` a `172.31.255.255` son 16 bloques. Es el error de examen más frecuente de la historia. - Uso restringido, no ruteable: `127.0.0.0/8` (loopback), `0.0.0.0/8`, `169.254.0.0/16` (link-local). Y la aritmética de alineación: **intacta**.

Cambió, o hay que reencuadrar: - **El `/48` por sitio final ya no se recomienda.** `RFC 6177` (2011) dejó de recomendar `/48` a cada sitio final; hoy los RIR asignan `/48` (a veces `/56`) **al operador** y el operador reparte. La aritmética no cambia; la justificación, sí. - **En IPv6 el problema desaparece:** el `/64` es la unidad de subred estándar (con SLAAC), así que VLSM en IPv6 es trivial. **VLSM es exclusivo de IPv4**, y decirlo en un examen es un punto. - **El `/20` del ejemplo ya no se compra.** RIPE NCC hizo su última asignación `/22` el **25 de noviembre de 2019**; hoy un `/20` se consigue por transferencia de mercado, no por asignación. - **El caso de uso actual es la red virtual privada del proveedor** (*VPC*, *overlay*): los rangos privados **se solapan entre inquilinos**, así que dos redes con `192.168.1.0/24` no se conectan sin traducción. Y en la casa no hay nada que subnetear: el CGNAT del ISP mete al cliente detrás de `100.64.0.0/10`.

> **Consecuencia para el examen:** la aritmética y el procedimiento **no se tocan nada**. Lo único a actualizar es la *justificación*: el bloque vino por transferencia y los rangos privados se solapan entre redes virtuales.



1.6 Plan de numeración completo: casa central y dos sucursales

Datos. Bloque público **`220.10.10.0/26`** (64 direcciones) para enlaces WAN y *pool* de traducción; hosts internos en **`192.168.0.0/16`**. Necesidades: LAN-CC-usuarios 250 hosts, LAN-S2 100, LAN-S1 50, LAN-CC-

servidores 30 — **más 1 dirección de router en cada una**.

Paso 1 y 2 — ordenar (de mayor a menor) y elegir prefijo.

```
251 → 2^8 − 2 = 254 ≥ 251 ✓              →  h = 8  →  /24  (512 dir)
101 → 2^7 − 2 = 126 ≥ 101 ✓              →  h = 7  →  /25  (256 dir)
 51 → 2^6 − 2 =  62 ≥  51 ✓              →  h = 6  →  /26  (128 dir)
 31 → 2^5 − 2 =  30 ≥  31  ✗  NO ALCANZA
      2^6 − 2 =  62 ≥  31 ✓              →  h = 6  →  /26  (128 dir)
```

⚠️ El `+2` (red + broadcast) va **dentro** del `2^h`, no afuera: por eso "31 direcciones → un `/27` de 30 utilizables" no alcanza y hay que subir a `/26`.

Paso 3 — asignación contigua y alineada en `192.168.0.0/16`.

```
/24  tamaño 256   inicio = 0 (múltiplo de 256 ✓)   →  192.168.1.0/24    puntero → 192.168.2.
/25  tamaño 128   inicio = 256 ✓  (256/128 = 2)    →  192.168.2.0/25    puntero → 192.168.2.
/26  tamaño  64   inicio = 384 ✓  (384/64 = 6)     →  192.168.2.128/26  puntero → 192.168.2.
/26  tamaño  64   inicio = 448 ✓  (448/64 = 7)     →  192.168.2.192/26  puntero → 192.168.3.

Consumo: 512 + 256 + 128 + 128 = 1024 de 65536.  Libre: 64512.
Mayor prefijo libre contiguo: 192.168.3.0/22 (4096 direcciones).
Gateways:  LAN-CC-usuarios .1.1 · LAN-S2 .2.1 · LAN-S1 .2.129 · LAN-CC-serv .2.193
```

Paso 3' — los enlaces públicos en `220.10.10.0/26`. Todos `/30`: los arranques 0, 4, 8, 12, 16 y 20 son múltiplos de 4, sin huecos.

```
220.10.10.0/30    Casa central ↔ ISP        casa .2    ISP .1
220.10.10.4/30    Casa central ↔ Sucursal 1 casa .5
220.10.10.8/30    Casa central ↔ Sucursal 2 casa .9
220.10.10.12/30   Pool de traducción NAT     R1 usa .13
220.10.10.16/30   Sucursal 1 ↔ ISP          sucursal .18   ISP .17
220.10.10.20/30   Sucursal 2 ↔ ISP          sucursal .22   ISP .21

Consumido 6 × 4 = 24 de 64.  Libre: .24 – .63 = 24/29 (8) + 32/27 (32) = 40.
```

Justificación de cada decisión.

Decisión	Por qué
Privado para hosts, público para enlaces	Los hosts nunca son alcanzables desde Internet: gastar dirección pública ahí es tirar el recurso. Los enlaces WAN sí: son lo que compra el ISP.
VLSM en vez de FLSM	Con FLSM las cuatro LAN serían `/24`: `4 × 512 = 2048` para 433 que necesito. Con VLSM: 1024.
`/30` para punto a punto	Un enlace entre routers necesita 2 direcciones; `/30` = 4 (2 útiles + red + broadcast). Existe `/31` para punto a punto 🔍 (creo que RFC 3021: **verificá**); en FING la respuesta esperada es `/30`.
Contiguo y alineado	Permite no perder direcciones **y** agregar rutas (§2.3).
R1 con dirección en cada LAN	Sin ella no hay a quién mandarle lo que no es local.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — VLSM con cuatro subredes de tamaños distintos

Enunciado. Del bloque privado `172.20.0.0/16`: LAN de ventas con **300** hosts, de producción con **120**, de contabilidad con **60**, y un enlace punto a punto con el Data Center. **(a)** Prefijos. **(b)** Asignación alineada. **(c)** Sobrantes. **(d)** Mayor prefijo libre contiguo.

Paso 1 — ordenar (de mayor a menor): 301 → 121 → 61 → 2. **Paso 2 — prefijo de cada una.**

```
301:  2^8 − 2 = 254 < 301 ✗   2^9 − 2 = 510 ≥ 301 ✓  →  h = 9  →  /23  (512 dir)
121:  2^7 − 2 = 126 ≥ 121 ✓                          →  h = 7  →  /25  (128 dir)
 61:  2^6 − 2 =  62 ≥  61 ✓                          →  h = 6  →  /26  ( 64 dir)
  2:  2^2 − 2 =   2 ≥   2 ✓                          →  h = 2  →  /30  (  4 dir)
```

Interpretación: cada prefijo es el **menor** que todavía alcanza; un `/24` para ventas habría servido (510 ≥ 301) pero habría tirado 211 direcciones de más.

Paso 3 — asignación con el puntero, verificando la alineación bloque por bloque (esto separa una respuesta de 4 puntos de una de 10):

```
puntero = 172.20.0.0
/23  inicio = 0 ✓                    →  172.20.0.0/23    puntero → 172.20.2.0
/25  inicio = 512 ✓  (512/128 = 4)   →  172.20.2.0/25    puntero → 172.20.2.128
/26  inicio = 640 ✓  (640/64 = 10)   →  172.20.2.128/26  puntero → 172.20.2.192
/30  inicio = 704 ✓  (704/4 = 176)   →  172.20.2.192/30  puntero → 172.20.2.196

172.20.0.0/16
├─ .0.0      /23   ventas        (512 dir, .0.0 – .1.255)
├─ .2.0      /25   producción    (128 dir, .2.0 – .2.127)
├─ .2.128    /26   contabilidad  ( 64 dir, .2.128 – .2.191)
├─ .2.192    /30   enlace P2P    (  4 dir, .2.192 – .2.195)
└─ .2.196 ──────────── LIBRE
```

(c) Direcciones sobrantes.

```
Consumidas:      512 + 128 + 64 + 4 = 708        Totales del /16: 65 536
Libres:          65 536 − 708 = 64 828      (98,9 % del bloque)
Desperdicio DENTRO de los bloques:
  /23: 510 util − 301 usadas = 209    /25: 126 util − 121 usadas =  5
  /26:  62 util −  61 usadas =   1    /30:   2 util −   2 usadas =  0  → 215
Las otras 708 − 215 = 493 son red + broadcast de las 4 redes.
```

(d) Mayor prefijo libre contiguo. El libre arranca en `172.20.2.196` (offset 708 dentro del /16). Para que un prefijo sea válido, ese offset tiene que ser múltiplo del tamaño, y no lo es para ningún tamaño ≥ 8 (`708/8 = 88,5`, `708/16 = 44,25`, `708/32 = 22,125`, `708/64 = 11,06`, `708/128 = 5,53`, `708/256 = 2,77`, `708/512 = 1,38` — ninguno entero). El primer arranque grande es `172.20.4.0` (offset 1024), y de ahí se encadenan `.4.0/23`, `.6.0/22`, `.10.0/21`, `.18.0/20`, `.34.0/19`, `.66.0/18`, hasta **`172.20.128.0/17`** (32768 dir), que es el mayor.

> Lo que **demuestra** que lo sabés no es el número grande: es la alineación de `172.20.128.0/17`. Un `/17` dentro de un `/16` ocupa `2^(24-17) = 128` unidades del tercer octeto, así que el tercer octeto tiene que ser `0` o `128`.

⚠️ **Errores típicos:** sumar el router dos veces (`+1` por red, aunque sea el mismo router); usar `2^h ≥ H` en vez de `2^h − 2 ≥ H`; y redondear `2^h − 2` hacia arriba cuando queda corto (caso de 31 direcciones).

2.2 Ejercicio 2 — *Maximally-useful packaging* desde un `/16`, y cuántas sobran

Enunciado. De `10.0.0.0/16` sacar **8 LAN de 1000 hosts**, **100 LAN de 100 hosts**, **100 LAN de 10 hosts** y **6 enlaces punto a punto**. **(a)** ¿Alcanza? **(b)** Plan. **(c)** Sobrantes y desperdicio.

(a) La cuenta de checksum va primero, porque si no alcanzara hay que subir prefijos y cambiar todo el plan. Prefijos: 1000 → `/22`, 100 → `/25`, 10 → `/28`, enlace → `/30`.

```
  8 LAN × 1024 dir  =  8 192
100 LAN ×  128 dir  = 12 800
100 LAN ×   16 dir  =  1 600
  6 enlaces ×   4   =     24
                      ________

TOTAL                        22 616  ≤  65 536   ✓  sobra 42 920
```

(b) Plan. Por el atajo de §1.3 no hay huecos: cada tamaño divide al anterior.

```
puntero = 10.0.0.0
8 × /22   =  8192 dir = 32 unidades de tercer octeto
           → 10.0.0.0 – 10.0.31.255      puntero = 10.0.32.0
100 × /25 = 12 800 dir = 50 unidades
           → 10.0.32.0 – 10.0.81.255    puntero = 10.0.82.0
100 × /28 =  1600 dir = 6 unidades + 64
           → 10.0.82.0 – 10.0.88.63     puntero = 10.0.88.64
6 × /30   =    24 dir
           → 10.0.88.64 – 10.0.88.87   puntero = 10.0.88.88
LIBRE: 42 920 dir

Alineación (offset dentro del /16 ÷ tamaño del bloque):
  10.0.32.0   →   8192 / 128 =   64 ✓      10.0.82.0  → 20992 / 16 = 1312 ✓
  10.0.88.64  →  22592 /   4 = 5648 ✓
```

(c) Sobrantes y desperdicio.

```
Libres: 65 536 – 22 616 = 42 920   (65,5 % del bloque)
Desperdicio dentro de los bloques:
   8 × (1022 – 1001) =    168
 100 × ( 126 –  101) =  2 500
 100 × (  14 –   11) =    300
   6 × (   2 –    2) =      0          →  2 968
Red + broadcast: 214 subredes × 2 = 428
Total no utilizable = 3 396
Eficiencia = (65 536 – 3 396) / 65 536 = 94,8 %
```

Interpretación: el 5,2 % que "se pierde" es casi todo red + broadcast (obligatorio) y el redondeo a potencia de 2 (inevitable). Con FLSM, las 214 redes tendrían que ser `/22` = 214 × 1024 = **219 136** direcciones, que no entran ni de lejos en el `/16`. **VLSM no es una optimización: es la diferencia entre que el plan exista y que no.**

2.3 Ejercicio 3 — El premio de alinear: agregación de rutas

Este es el motivo **real** por el que la alineación importa, y es lo que conecta esta clase con la de tablas de enrutamiento.

Regla. Dos o más prefijos se pueden resumir en uno solo si y solo si: son **contiguos**, la cantidad total de direcciones es **potencia de 2**, y el inicio del primero está **alineado** a ese tamaño.

```
.0/25 + .128/25  = 192.168.0.0/24  ✓   .0/24 + .1/24 = 192.168.0.0/23  ✓
192.168.1.0/24 + 192.168.2.0/24  →  ¡se crearían 512 = un /23!
   ¿Dónde arrancaría? En un múltiplo de 512 del tercer octeto:
      0 → 192.168.0.0/23  ✗ incluye 192.168.0.0/24, que no es nuestra
      2 → 192.168.2.0/23  ✗ incluye 192.168.3.0/24, que no es nuestra
   En AND: 1 = 00000001 AND /23 = 11111110 → 00000000 = 0
   → el resumen "natural" es 192.168.0.0/23, que INCLUYE una red ajena.
```

Conclusión: dos redes contiguas de 256 **no** se pueden resumir. Por eso en el plan de §1.6 puse las cuatro LAN alineadas a partir de `192.168.1.0`: así `192.168.2.0/25` + `.128/26` + `.192/26` se resumen en `192.168.2.0/24` (1 ruta en vez de 3), y `192.168.1.0/24` + `192.168.2.0/24` en `192.168.0.0/23` (1 ruta en vez de 2). **La agregación de rutas es un criterio de diseño del plan**, no una consecuencia: es lo que dice la oficial del `2010-12 P2a-i` (§3.2.1) al elegir el plan que deja `LAN3 + LAN4` sumarizables en un solo `/24`.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen `2011-03`, Problema 2 d — dividir un `/24` para 70 puestos y 15 servidores

> **Fuente:** Examen `eirc1103` / `sirc1103`, 21 de febrero de 2011, Problema 2 (30 puntos), apartado d.
> **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

Enunciado (literal)

> Se adquiere para la Red A el bloque de direcciones IP públicas `100.100.100.0/24`, y los administradores deciden aislar la red de servidores de los puestos de trabajo. Especifique las subredes del bloque mencionado que cumplan con este requerimiento y permitan dejar la mayor cantidad posible de direcciones IP para uso futuro. Mencione las modificaciones necesarias en el router R1 y su nueva tabla de enrutamiento.

Contexto. Red A: LAN Ethernet privada con **15 servidores** y **70 puestos**; R1 hace NAT con `220.10.10.30` (enlace `220.10.10.28/30`) para llegar a `219.41.31.21`. Ahora compran `100.100.100.0/24` — **público** — y quieren **aislar** servidores de puestos.

Solución oficial

> **Subred para 70 puestos de trabajo:** 70 direcciones IP + 1 para el router R1 (Además siempre se deben considerar la direcciones de red y de broadcast) → `100.100.100.0/25` **Subred para 15 servidores:** 15 direcciones IP + 1 para el router R1 → `100.100.100.128/27` El router R1 se conecta a cada subred y no es necesario que implemente NAT, ya que las estaciones de trabajo y los servidores tienen direcciones IP públicas. **Nueva tabla de enrutamiento de R1:** `100.100.100.0/25` → C · `100.100.100.128/27` → C · `220.10.10.28/30` → C · `0.0.0.0/0` → `220.10.10.29`.

Mi resolución, con la aritmética completa

Paso 1 — ordenar: 70 puestos, después 15 servidores. **Paso 2 — los prefijos.**

```
PUESTOS:    70 hosts + 1 (gateway R1) = 71
   2^6 - 2 =  62   <  71    ✗   NO ALCANZA
   2^7 - 2 = 126   ≥  71    ✓   →  h = 7  →  /25  (128 dir, 126 utilizables)

SERVIDORES: 15 hosts + 1 (gateway R1) = 16
   2^4 - 2 =  14   <  16    ✗   NO ALCANZA   ← ¡el error más tentador!
   2^5 - 2 =  30   ≥  16    ✓   →  h = 5  →  /27  ( 32 dir,  30 utilizables)
```

⚠️ **El error que casi todos cometen** es hacer `16 → 2^4 → /28`, pensando que 16 es potencia de 2. Pero `2^4 = 16` es el total **incluyendo** red y broadcast: los utilizables son 14, y necesito 16. El `+2` va **adentro**.

Paso 3 — asignación contigua y alineada. `puntero = 100.100.100.0`

```
/25  tamaño 128   inicio = menor múltiplo de 128 ≥ 0    = 0     → 100.100.100.0/25
      (0 es múltiplo de 128 ✓)                              (.0 – .127)   puntero → .128
/27  tamaño  32   inicio = menor múltiplo de  32 ≥ 128  = 128   → 100.100.100.128/27
      (128 es múltiplo de 32 ✓)                             (.128 – .159) puntero → .160

100.100.100.0/24
├─ .0   /25   PUESTOS     red .0   | bc .127 | hosts .1   – .126 | gw .1
├─ .128 /27   SERVIDORES  red .128 | bc .159 | hosts .129 – .158 | gw .129
└─ .160 ──────── LIBRE (96 dir) = 100.100.100.160/26 (64) + 100.100.100.224/27 (32)
```

Modificaciones necesarias en R1.

1. **Dos interfaces** (o dos sub-interfaces de VLAN) en vez de una: una por subred, porque ahora están **separadas**.
2. **Se elimina la traducción NAT.** Los hosts ya son alcanzables por dirección pública; traducirlos "sobre-vendería" las públicas que se acaban de comprar para eso. La solución oficial lo dice textualmente.
3. **La ruta por defecto no cambia:** `0.0.0.0/0` → `220.10.10.29` por `220.10.10.28/30`, y cada subred **conserva su gateway** (`100.100.100.1` y `100.100.100.129`).
4. **El matiz que vale puntos:** el enunciado dice "aislar la red de servidores de los puestos", y dos subredes distintas **no** aíslan nada por sí solas — R1 reenvía de una a otra salvo que filtre. Lo que se entiende es aislamiento de **dominio de broadcast y de numeración**, no cortafuegos; si se hubiera pedido aislamiento de seguridad, la respuesta sería ACL o cortafuegos.

La aritmética que hay que mostrar: ¿esto **maximiza** el uso?

Plan	Puestos	Servidor	Total	Libre del /24
OFICIAL /25 + /27	128	32	160	96 (37,5 %)
"Cómodo" /25 + /26	128	64	192	64 (25,0 %)
"Apretado"/25 + /28	128	14 < 16	inválido	no sirve

Desperdicio interno: oficial (126−71)+(30−16) = 69 ; "cómoda" (126−71)+(62−16) = 101

La conclusión honesta:

> **Nota: la solución oficial dice `/25` y `/27`. Verifiqué la cuenta y es correcta — y además es la *única* solución posible.** Los dos prefijos están forzados: `/26` no alcanza para 71 utilizables y `/28` no alcanza para 16. No hay margen de decisión en los prefijos, solo en el **orden** de asignación.
>
> **Mi lectura difiere en el enfoque, y es lo que hace interesante el enunciado.** "Dejar la mayor cantidad posible para uso futuro" tiene dos readings (§1.4). La oficial satisface la A (gasta 160 de 256, el mínimo posible) **y también** la B: el remanente `100.100.100.160` – `100.100.100.255` son **96 direcciones contiguas que se descomponen limpiamente en `100.100.100.160/26` + `100.100.100.224/27`**, justo lo que hace falta para dos LANs futuras de hasta 62 y de hasta 30 hosts.

> Con `/26` para los servidores, en cambio, el libre quedaría partido en `.160/28` + `.176/27` + `.208/27` = 8 + 32 + 32: tres bloques distintos, mucho menos útil.
>
> O sea: **lo que suma puntos no es "la oficial está bien", sino "la oficial es correcta porque el mínimo de cada prefijo la obliga, y además produce un remanente alineado que sirve para el segundo reading del objetivo"**.

⚠️ **Errores típicos:** `/28` para los servidores; no contar el router (`+1` en cada subred); dejar el NAT puesto; olvidar la entrada `0.0.0.0/0` en la nueva tabla (sin ella los hosts de las subredes nuevas no salen a Internet, que era el objetivo de la red A).

3.2 Dos apuntes de examen que conviene tener resueltos

3.2.1 `147.10.8.0/21` con hasta 120 hosts por LAN

> **Fuente:** Examen 2008-07 (`sirc0807`, Problema 2 a-i); el banco lo indexa también bajo `sirc1007`. Se resuelve **entero** en la Clase 29.

```
Enunciado: "Realice un plan de numeración de las subredes LAN3, 4 y 5
utilizando 147.10.8.0/21, asumiendo que no se conectan más de 120 hosts por
subred. ¿Cuántas subredes se pueden numerar?"

Solución oficial: "Para cada subred se necesitan como máximo 120 + 1 + 2 = 123
direcciones IP por LAN."  →  2^6 = 64 < 125 ✗ ;  2^7 = 128 ≥ 125 ✓  →  /25
¿Cuántas subredes?  2^(32-21) = 2048 dir ; 2048 / 128 = 16    [o: 11 − 7 = 4 bits → 2^4]

La oficial da dos planes posibles y NO son equivalentes:
  PLAN A:  LAN3 147.10.8.0/25 · LAN4 147.10.8.128/25 · LAN5 147.10.9.0/25
           LAN3 + LAN4 = 147.10.8.0/24  ← SUMARIZA (0 es múltiplo de 256)
           ⇒ R2 necesita 2 rutas hacia la empresa.
  PLAN B:  LAN3 147.10.8.0/25 · LAN4 147.10.9.0/25 · LAN5 147.10.10.0/25
           NO se puede sumarizar: resumir LAN4 (.9.0–.9.127) y LAN5
           (.10.0–.10.127) exige arrancar en .8.0 o en .10.0, y .9.0 no es
           ninguno de los dos.  ⇒ R2 necesita 3 rutas.
```

> **El `+2` de la oficial:** 120 hosts + 1 router + 2 de **reserva** para crecimiento inmediato de cada LAN (no red ni broadcast, que ya están dentro del `2^h`). Hay que decir de dónde sale, porque si no parece un error de suma.
>
> **Conclusión para el examen:** *"la solución no es única, pero no todas las soluciones son equivalentes: el plan tiene que optimizar no solo el uso de direcciones sino también la cantidad de rutas"* — y eso se decide en el paso 3 (asignación alineada), no en el paso 2 (elegir prefijo).

3.2.2 `192.168.2.0/23` con hasta 250 hosts por subred

> **Fuente:** Examen 2010-07 (`sirc1007`, Problema 2 a-i), resuelto **entero** en la Clase 29 (Ejercicio 4).

```
250 hosts + 1 router = 251    + red y broadcast  ⇒  el bloque necesita ≥ 253
2^7 = 128 < 253 ✗  (/25 NO alcanza — 128, no 256)
2^8 = 256 ≥ 253 ✓  →  h = 8  →  /24   (254 utilizables, sobran 3)

¿Cuántas subredes entran en el /23?
   Por bits:   /23 deja 9 bits de subred; /24 usa 8; quedan 1  →  2^1 = 2
   Por tamaño: 2^(32-23) = 512 direcciones ; 512 / 256 = 2  ✓
Asignación:  192.168.2.0/24 (LAN2, gw .2.1) · 192.168.3.0/24 (LAN3, gw .3.1)
Cuentas:     512 consumidas de 512  →  LIBRE: 0
             no utilizable = 2 × 2 (red+bc) + 2 × 3 (sobrantes) = 10  (1,95 %)
```

El comentario de diseño que vale puntos: el bloque queda **al 100 % de ocupación**; si la empresa supiera que iba a necesitar una tercera sucursal debería haber pedido un `/22`, pero el enunciado no lo dice y la respuesta correcta es usar el `/23` entero.

3.3 Examen `2006-08`, Pregunta 4 (10 puntos) — dividir un `/25` en 18, 11, 9 hosts y 3 enlaces

> **Fuente:** Examen 5 de agosto de 2006, `sirc0608`, Pregunta 4: un solo bloque, cinco necesidades de tamaños distintos y la pregunta por las redes libres. Se resuelve **entero** en la Clase 29; acá va la aritmética:

```
Bloque:  200.40.224.0/25  →  32 − 25 = 7 bits de host
         Total 2^7 = 128 direcciones  ·  utilizables 128 − 2 = 126

18 hosts → 20 dir ;  2^4 = 16 < 20 ✗ ;  2^5 = 32 ≥ 20 ✓  →  /27 (32 dir)
11 hosts → 13 dir ;  2^4 = 16 ≥ 13 ✓                      →  /28 (16 dir)
 9 hosts → 11 dir ;  2^3 =  8 < 11 ✗ ;  2^4 = 16 ≥ 11 ✓  →  /28 (16 dir)
enlace  →  4 dir ;  2^2 =  4 ≥  4 ✓                      →  /30 ( 4 dir)

Plan oficial:  .0/27  ·  .32/28  ·  .48/28  ·  .64/30  ·  .68/30  ·  .72/30
Consumido:     32 + 16 + 16 + 4 + 4 + 4 = 76    →  libre 128 − 76 = 52
Oficial: 7 bloques libres = .76/30 (4) + seis /29 de 8 = 4 + 48 = 52
Mía:     4 bloques = .76/30 + .80/28 + .96/28 + .112/28 = 4 + 16×3 = 52
   (misma superficie, menos entradas de tabla)
⚠️  .80/27 es IMPOSIBLE: 80/32 = 2,5 no es entero. Esa es la trampa.
```

PARTE 4 — CIERRE

📌 Resumen

1. **FLSM** obliga a que todas las subredes tengan el tamaño de la mayor; **VLSM** le da a cada una lo que necesita.
2. Con FLSM, 1000 + 100 hosts desde un `/20` desperdicia **946** direcciones; con VLSM, **50**.
3. Procedimiento: **ordenar de mayor a menor** → `2^h − 2 ≥ H` → **asignar contiguo y alineado** → **llevar el puntero**.
4. **Alineado** es `dir_de_red mod tamaño == 0`; sin eso los bloques se superponen: `.160/26` es en realidad `.128/26`.
5. De mayor a menor cada tamaño divide al anterior, así que **no hay huecos**.
6. "Dejar lo máximo para el futuro" tiene dos readings — **gastar menos hoy** y **dejar el remanente más limpio** — y se responden juntos.
7. `/25 + /27` para 70 puestos y 15 servidores (`2011-03 d`): la oficial es correcta **y es la única posible**; remanente de 96 = `/26 + /27`.
8. `147.10.8.0/21` con 123 por LAN → `/25` → **16 subredes**; el plan decide si R2 tiene **2** rutas o **3**.
9. `192.168.2.0/23` con 251 por LAN → `/24` → **2 subredes exactas**, bloque al 100 %.
10. `200.40.224.0/25` con 18+11+9 y 3 enlaces → `/27 + /28 + /28 + 3×/30`, libres 52 direcciones.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué ordenar "de mayor a menor" es obligatorio y no cosmético?

<details>
<summary>Respuesta</summary>
</details>

2. Alguien asigna `192.168.1.160/26`. Demostrá con la cuenta que no es una dirección de red válida.

<details>
<summary>Respuesta</summary>
</details>

3. 70 puestos + 1 router y 15 servidores + 1 router, desde un `/24`: ¿qué prefijos salen y por qué no `/28` para los servidores?

<details>
<summary>Respuesta</summary>
</details>

4. De un `/20` (4096 dir) se necesitan LANs de 1000 y de 100 hosts, más un router en cada una. ¿Cuánto desperdicia FLSM y cuánto VLSM?

<details>
<summary>Respuesta</summary>
</details>

5. ¿Cuántas subredes `/25` entran en `147.10.8.0/21`? Las dos cuentas.

▶ Respuesta

6. `192.168.1.0/24` y `192.168.2.0/24` son contiguas y suman 512 direcciones. ¿Se pueden resumir?

▶ Respuesta

7. En el plan de la empresa, las cuatro LAN quedaron en `192.168.1.0/24`, `192.168.2.0/25`, `192.168.2.128/26` y `192.168.2.192/26`. ¿Cuántas rutas ahorra esa alineación frente a un plan "desalineado" de `.1.0/24`, `.3.0/24`, `.4.0/24`, `.5.0/24`?

▶ Respuesta

8. En `2010-12 P2a-i`, ¿por qué la oficial prefiere `LAN3 .8.0/25 · LAN4 .8.128/25 · LAN5 .9.0/25` y no `LAN3 .8.0/25 · LAN4 .9.0/25 · LAN5 .10.0/25`?

▶ Respuesta

🔗 Referencias

- **`eirc1103` / `sirc1103`** (21-feb-2011) — **Problema 2 d**: dividir `100.100.100.0/24` para 70 puestos + 15 servidores (§3.1) · P2 a-c son las Clases 30 y 31
- **`sirc0807`** (1-ago-2008) — P2 a-i: `147.10.8.0/21` con 120 hosts por LAN (§3.2.1)
- **`sirc1007`** (27-jul-2010) — P2 a-i: `192.168.2.0/23` con 250 hosts (§3.2.2)
- **`sirc0608`** (5-ago-2006) — P4: `200.40.224.0/25` en 18, 11 y 9 hosts + 3 enlaces (§3.3) · P2: por qué ARP (Clase 30)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 "La capa de red"
- **RFC 1918** (privadas) 🔍 · **RFC 4632** (CIDR) 🔍 · **RFC 3021** (/31 P2P) 🔍 · **RFC 6177** (2011) · **RFC 8200** (2017)
- **`ACTUALIZACIONES.md` §5** — agotamiento del IPv4 (RIPE NCC, 25-nov-2019) y CGNAT
- **Clase siguiente:** 29 — Taller de subnetting y planes de numeración